

BÁO CÁO KẾT QUẢ THỰC TẬP

Tuần 4: Chuẩn hóa Xác thực với Passport và Phân quyền Tài khoản nâng cao

I. LÝ THUYẾT

1. Hệ thống xác thực Passport và Chiến lược mã báo (JWT Strategy)

- **Passport:** Thư viện trung gian quản lý xác thực chuẩn cho các ứng dụng Node.js. Thay vì tự bóc tách chuỗi mã xác thực (Token) bằng tay, Passport cung cấp cấu trúc rõ ràng giúp tự động hóa quá trình kiểm tra thông tin đăng nhập một cách bảo mật.
- **Chiến lược JWT:** Một phần mở rộng của Passport giúp tự động trích xuất mã Token từ tiêu đề gửi lên, giải mã và xác minh tính toàn vẹn của Token trước khi cho phép người dùng đi tiếp vào hệ thống.

2. Cơ chế kiểm soát truy cập dựa trên vai trò

- **Gắn nhãn quyền bằng Decorator:** Phương pháp sử dụng các nhãn đánh dấu (Metadata) đặt ngay trên đầu các hàm xử lý API để quy định rõ chức năng này yêu cầu quyền gì (Ví dụ: **ADMIN** hay **USER**).
- **Bộ lọc chặn phân quyền (RolesGuard):** Lớp bảo vệ đóng vai trò như bảo vệ kiểm tra thẻ. Nó đối chiếu vai trò thực tế của người dùng vừa đăng nhập với yêu cầu của API để quyết định cho qua hoặc trả về lỗi từ chối truy cập.

II. NỘI DUNG THỰC HÀNH & CÁC CÂU LỆNH CẤU HÌNH

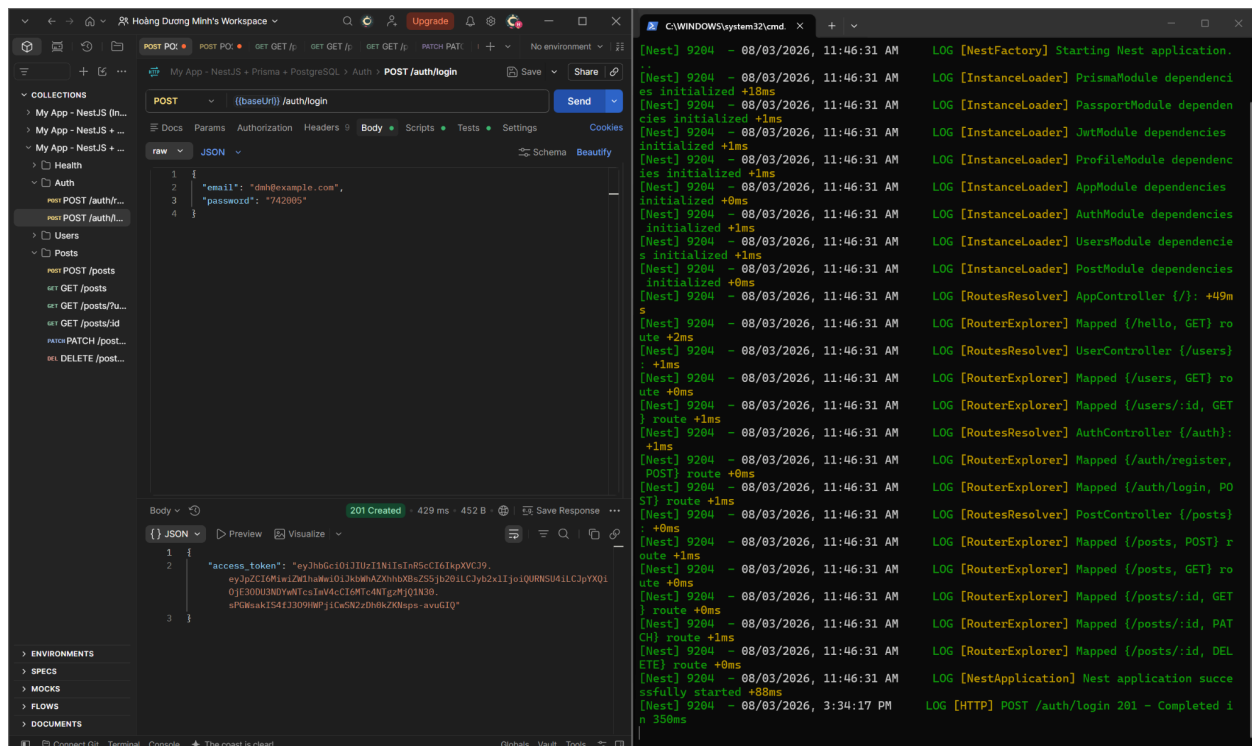
1. Nội dung thực hành

- Cài đặt và tích hợp bộ thư viện Passport, Passport-JWT để thay thế cho bộ kiểm tra token tự viết cũ.
- Xây dựng cấu trúc tệp tin **jwt.strategy.ts** để tự động hóa luồng giải mã Token và xác minh sự tồn tại của người dùng dưới database.
- Viết custom decorator **@Roles()** và xây dựng **RolesGuard** để triển khai bộ lọc chặn phân quyền toàn cục.
- Cập nhật logic điều phối tại các Controller: Áp dụng kiểm tra vai trò chặt chẽ (Ví dụ: chỉ **ADMIN** mới có thể sửa đổi cấu trúc lớn hoặc xóa các nội dung nhạy cảm).
- Toàn bộ mã nguồn tuần này được đẩy lên GitHub trong thư mục week4.

2. Câu lệnh cấu hình cài đặt bổ sung

Mở Terminal tại thư mục gốc của dự án và chạy lệnh sau để bổ sung các module của Passport:

```
npm install @nestjs/passport passport passport-jwt @nestjs/jwt
npm install --save-dev @types/passport-jwt
```



1.2. Kiểm thử API yêu cầu xác thực khi không gửi kèm Token (POST /posts)

- Mô tả:** Khi client cố tình gửi request tạo bài viết mới mà không đính kèm thẻ thông hành trong Header, `AuthGuard('jwt')` của Passport phát hiện trạng thái trống, từ chối xử lý và trả về mã lỗi xác thực chuẩn.

The screenshot displays a REST client interface with a collection named 'POST /posts'. The request is configured with the following details:

- Method:** POST
- URL:** {{baseUrl}}/posts
- Auth Type:** Bearer Token
- Token:** (Empty field)

The response is a 401 Unauthorized status, indicating that the request was rejected due to missing or invalid authentication credentials.

```
{
  "message": "Unauthorized",
  "statusCode": 401
}
```

The background shows a terminal window with application logs, including messages like 'LOG [InstanceLoader] JwtModule dependencies initialized' and 'LOG [RouterExplorer] Mapped {/posts, POST} route', providing context for the application's state during the test.

1.3. Kiểm thử API yêu cầu xác thực khi CÓ gửi kèm Token hợp lệ (POST /posts)

- **Mô tả:** Client cấu hình **Bearer Token** bằng chuỗi JWT hợp lệ. **AuthGuard ('jwt')** chấp thuận, **JwtStrategy** thực hiện giải mã signature thành công, tự động trích xuất thông tin định danh và nạp thẳng vào **req.user**. Bài viết được tạo thành công gắn với ID tài khoản người gửi.

The screenshot displays a development environment with VS Code and a terminal window. In VS Code, a REST client request is configured for the endpoint `POST /posts` with the following details:

- Auth Type:** Bearer Token
- Token:** A masked JWT token.
- Body:** A JSON object representing a post creation request.

```
{
  "id": 8,
  "title": "My Third Post",
  "content": "This post is connected via n-n relation schema using Prisma ORM.",
  "userId": 2
}
```

The terminal window shows the application logs, indicating that the request was successful and the post was created. The logs include the following entries:

```
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [InstanceLoader] PassportModule dependencies initialized +0ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [InstanceLoader] JwtModule dependencies initialized +1ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [InstanceLoader] ProfileModule dependencies initialized +1ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [InstanceLoader] AppModule dependencies initialized +1ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [InstanceLoader] AuthModule dependencies initialized +0ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [InstanceLoader] UsersModule dependencies initialized +1ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [InstanceLoader] PostModule dependencies initialized +0ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [RoutesResolver] ApplicationController (/): +65ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [RouterExplorer] Mapped (/hello, GET) route +0ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [RoutesResolver] UserController (/users): +0ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [RouterExplorer] Mapped (/users, GET) route +1ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [RoutesResolver] AuthController (/auth): +0ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [RouterExplorer] Mapped (/auth/register, POST) route +1ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [RouterExplorer] Mapped (/auth/login, POST) route +0ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [RoutesResolver] PostController (/posts): +0ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [RouterExplorer] Mapped (/posts, POST) route +1ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [RouterExplorer] Mapped (/posts, GET) route +0ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [RouterExplorer] Mapped (/posts/:id, GET) route +0ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [RouterExplorer] Mapped (/posts/:id, PATCH) route +1ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [RouterExplorer] Mapped (/posts/:id, DELETE) route +0ms
[Nest] 16244 - 08/03/2026, 3:41:01 PM LOG [NestApplication] Nest application successfully started +84ms
[Nest] 16244 - 08/03/2026, 3:56:48 PM LOG [HTTP] POST /posts 201 - Completed in 222ms
[Nest] 16244 - 08/03/2026, 3:58:34 PM LOG [HTTP] POST /posts 401 - Completed in 2ms
[Nest] 16244 - 08/03/2026, 3:59:04 PM LOG [HTTP] POST /posts 201 - Completed in 141ms
```

2. QUY TRÌNH PHÂN QUYỀN

2.1. Kiểm thử quyền ADMIN truy cập tài nguyên quản trị (GET /users)

- **Mô tả:** Tài khoản mang vai trò **ADMIN** (Ví dụ: id = 2) sử dụng mã token của mình để gọi endpoint lấy danh sách. Request vượt qua lớp xác thực, tiếp tục đi vào **RolesGuard**. Guard đối chiếu metadata thấy trùng khớp với nhãn **@Roles ('ADMIN')**, cho phép Controller và Service thực thi để truy xuất toàn bộ dữ liệu.

The screenshot displays a REST client interface on the left and application logs on the right.

REST Client Details:

- Method:** GET
- URL:** ((baseUrl)) /users
- Auth Type:** Bearer Token
- Token:** {{accessToken}}
- Status:** 200 OK
- Response Body (JSON):**

```
[{"id": 3, "name": "Trinh Duc T", "email": "tdt@example.com", "password": "$2b$12$RmGzZ6tM3RmBY4v..JDyevqPwIRNy..EWBk1x1CfIVs4QzGtcK", "role": "USER", "profile": {"id": 1, "bio": "Hello world, I am a software engineering student.", "avatar": "https://example.com/avatar.png", "userId": 3}}, {"id": 1, "title": "My First Post", "content": "This post is connected via 1-n relation schema using Prisma ORM.", "userId": 3}]
```

Application Logs:

- LOG [InstanceLoader] AppModule dependencies initialized +1ms
- LOG [InstanceLoader] AuthModule dependencies initialized +0ms
- LOG [InstanceLoader] UsersModule dependencies initialized +1ms
- LOG [InstanceLoader] PostModule dependencies initialized +0ms
- LOG [RoutesResolver] ApplicationController (/): +65ms
- LOG [RouterExplorer] Mapped (/hello, GET) route +4ms
- LOG [RouterExplorer] Mapped (/users, GET) route +0ms
- LOG [RouterExplorer] Mapped (/users/:id, GET) route +1ms
- LOG [RoutesResolver] AuthController (/auth): +0ms
- LOG [RouterExplorer] Mapped (/auth/register, POST) route +1ms
- LOG [RouterExplorer] Mapped (/auth/login, POST) route +0ms
- LOG [RoutesResolver] PostController (/posts): +0ms
- LOG [RouterExplorer] Mapped (/posts, POST) route +1ms
- LOG [RouterExplorer] Mapped (/posts, GET) route +0ms
- LOG [RouterExplorer] Mapped (/posts/:id, GET) route +0ms
- LOG [RouterExplorer] Mapped (/posts/:id, PATCH) route +1ms
- LOG [RouterExplorer] Mapped (/posts/:id, DELETE) route +0ms
- LOG [NestApplication] Nest application successfully started +84ms
- LOG [HTTP] POST /posts 201 - Completed in 222ms
- LOG [HTTP] POST /posts 401 - Completed in 141ms
- LOG [HTTP] POST /posts 401 - Completed in 1ms
- LOG [HTTP] POST /auth/login 201 - Completed in 228ms
- LOG [HTTP] GET /users 200 - Completed in 225ms

2.2. Kiểm thử quyền USER thường cố tình truy cập tài nguyên ADMIN (GET /users)

- **Mô tả:** Tài khoản mang vai trò **USER** (Ví dụ: id = 3) gửi request lấy toàn bộ danh sách thành viên. Sau khi xác thực danh tính xong, **RolesGuard** quét và đối chiếu thấy **req.user.role** là '**USER**', không thỏa mãn nhãn quản trị. Hệ thống chặn luồng xử lý tại chỗ và đưa ra lỗi từ chối ngay lập tức để bảo vệ dữ liệu.

The screenshot displays a REST client interface on the left and a terminal window on the right. The REST client shows a GET request to `/users` with a Bearer Token authentication. The response is a 403 Forbidden status with the following JSON body:

```
{
  "message": "You do not have permission to access this resource.",
  "error": "Forbidden",
  "statusCode": 403
}
```

The terminal window on the right shows application logs for a NestJS application. The logs indicate that the application successfully started and then received a GET request to `/users` at 3:41:01 PM. The response is a 403 Forbidden status, which is consistent with the REST client response.

2.3. Kiểm thử quy trình ngăn chặn IDOR khi Xóa bài viết (DELETE /posts/:id)

- **Kịch bản:** User thường (id = 3) sở hữu bài viết số 1 và số 5. Admin (id = 2) sở hữu bài viết số 2 và số 4. Tiến hành kiểm thử dùng tài khoản User thường để xóa bài viết số 4 (của Admin).
- **Mô tả:** Quyết định xử lý được đưa xuống dịch vụ. Tại đây, hàm `remove()` tiến hành đối chiếu kết hợp: biến `userRole` sau khi được chuyển đổi chữ hoa không phải là `'ADMIN'`, đồng thời `post.userId` (bằng 2) hoàn toàn khác biệt với `currentUser.id` (bằng 3). Điều kiện bảo mật kích hoạt, ngăn chặn hành vi xóa tài nguyên và trả về mã lỗi 403.

The screenshot displays a REST client interface with a collection of API endpoints. The selected endpoint is `DELETE /posts/:id`. The request body is a JSON object:

```
{ 1: { 2: "message": "You are not allowed to delete this post.", 3: "error": "Forbidden", 4: "statusCode": 403 5: }
```

. The response is a `403 Forbidden` status. The right pane shows a log of API calls, including `LOG [RouterExplorer] Mapped (/posts/:id, PATCH) route +1ms`, `LOG [RouterExplorer] Mapped (/posts/:id, DELETE) route +0ms`, `LOG [NestApplication] Nest application successfully started +66ms`, and `LOG [HTTP] DELETE /users/1 403 - Completed in 199ms`.